

AHB Example AMBA SYstem - ARM DDI 0170A Addendum 03

This addendum document details the AHB BusMatrix, which is an additional component in *Chapter 4 AHB Modules* in the *AHB Example AMBA SYstem (EASY)* Technical Reference Manual.

Text additions

Page	Insert location	Inserted text
4-84	Insert after final paragraph	See attached text

AHB BusMatrix

The BusMatrix allows parallel access to a number of shared AHB slaves from a number of different AHB masters. The switch determines which slave a particular master requires access to, and routes the input port to the appropriate slave.

The BusMatrix can be used in various different ways. The generic case is shown in Figure 1.

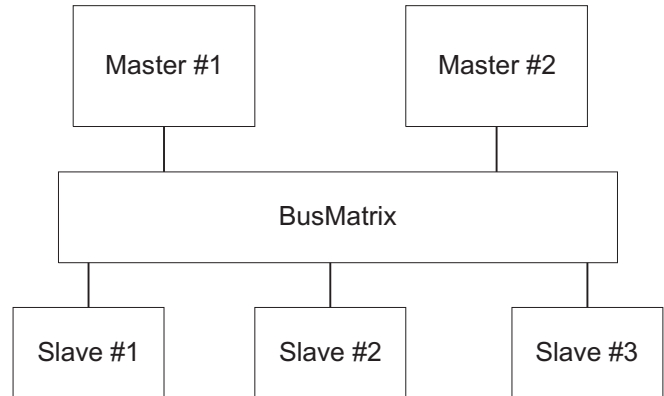


Figure 1 Generic BusMatrix

There is no restriction on input or output ports on the BusMatrix having just one master or slave. Figure 2 shows an example of how multiple masters or slaves can be connected to the BusMatrix.

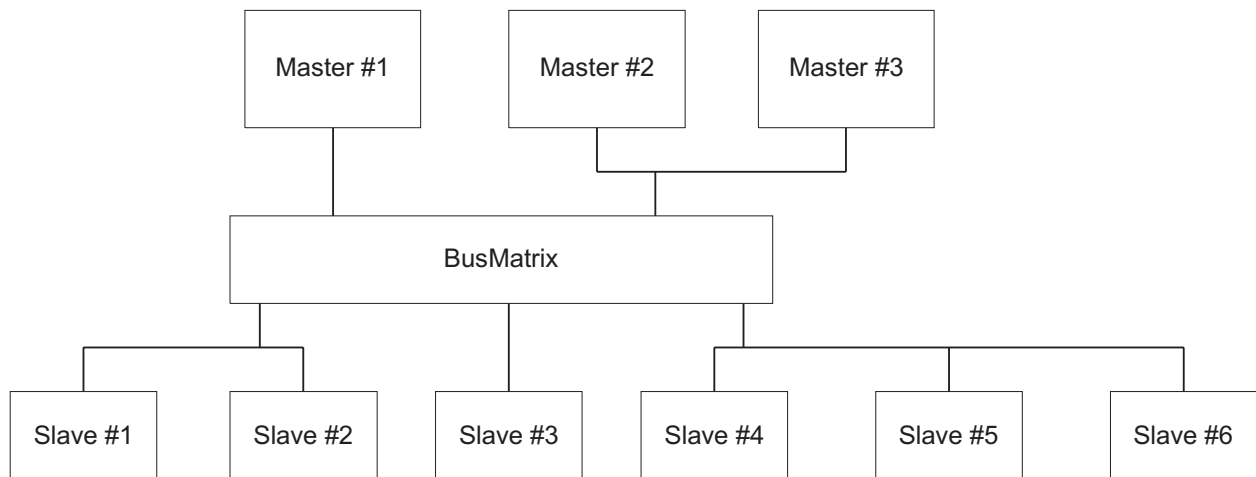


Figure 2 Complex use of BusMatrix

Example configurations

The following diagrams show more typical system configurations. Figure 3 shows:

- two bus masters
- an ARM processor and a DMA controller, both of which have shared access to an APB
- a block of on-chip memory
- an external memory interface.

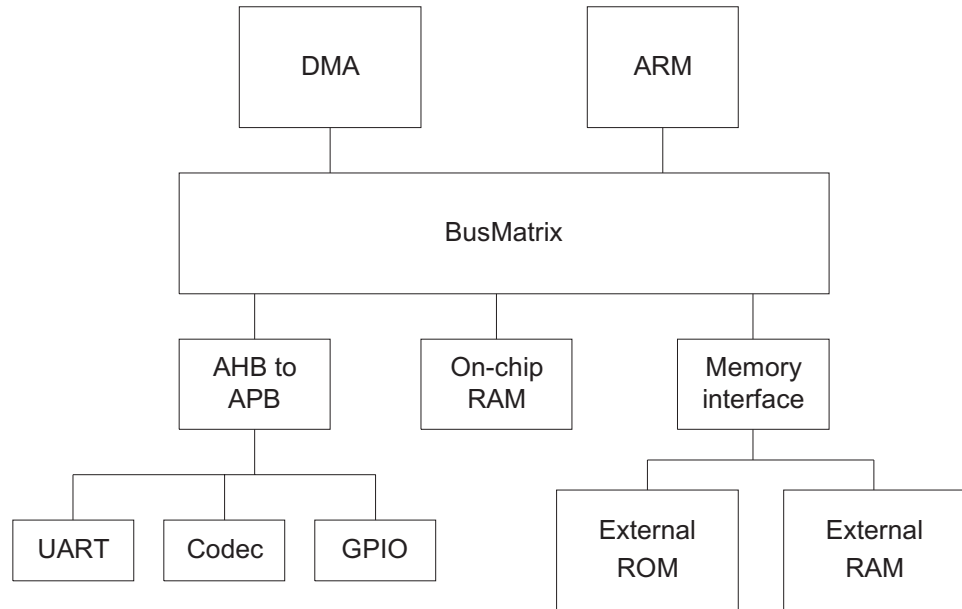


Figure 3 Example system using BusMatrix

The second example, in Figure 4, shows how a simple BusMatrix, with two input ports and a single slave port, can be used to share a slave in the system. This example shows how an on-chip RAM can be shared between two bus systems allowing message packets to be passed between them.

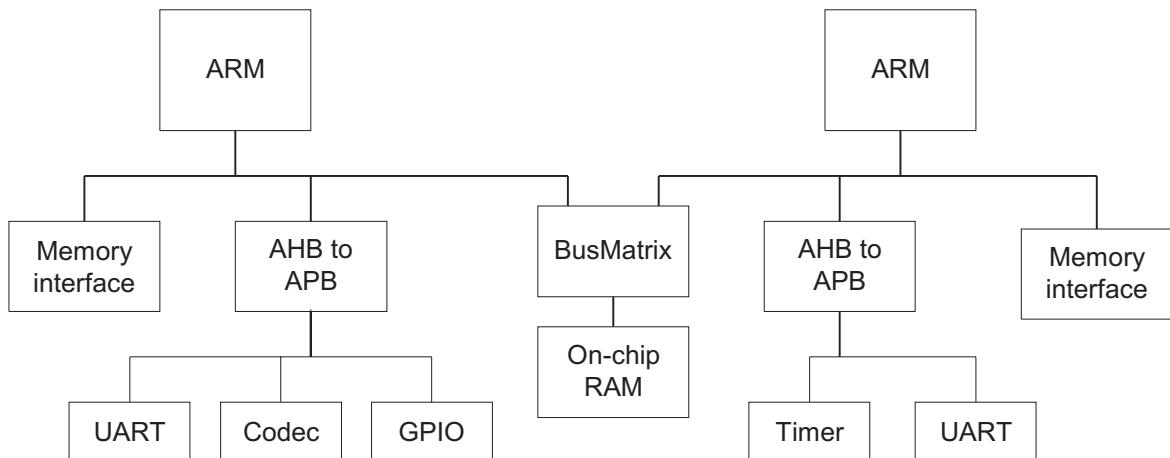


Figure 4 Example system using BusMatrix to connect subsystems

Functional description

Figure 5 shows a block diagram of a four input port and three output port BusMatrix.

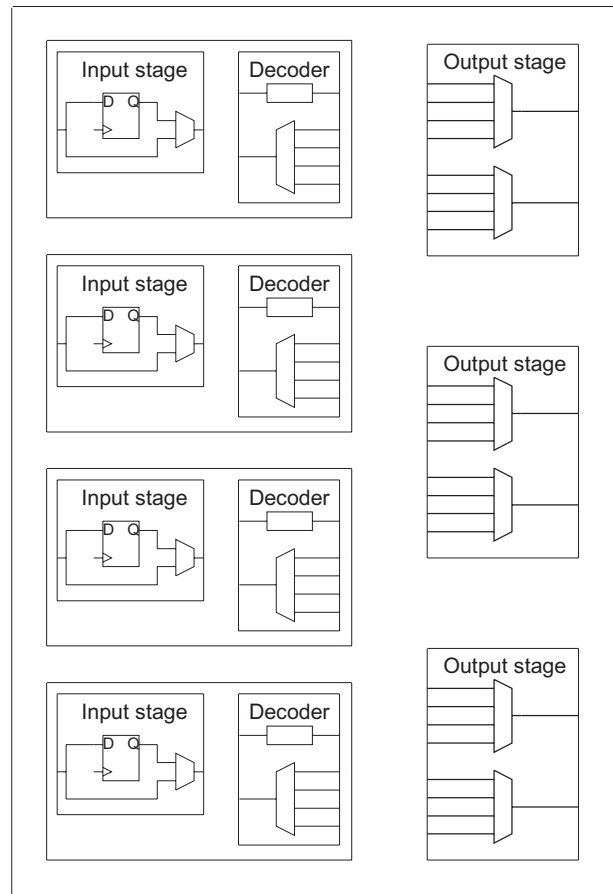


Figure 5 BusMatrix block diagram

The BusMatrix is made up of the following sections.

Input stage

The input stage provides the following functions:

- It registers and holds an incoming transfer if the slave that it is destined for is not able to accept the address of the transfer.
- If the BusMatrix changes between input ports in the middle of a burst, the input stage modifies the **HTRANS** and **HBURST** signals so that when the input port is used again, the remaining transfers in the burst meet the AHB specification.

Decode stage

Each input stage has an associated decode stage that determines which of the connected slaves a given transfer is destined for. During the address phase of a transfer, the decoder looks at the address and asserts the select signal to the appropriate output stage. Also, during the address phase the decoder routes the active signal from the appropriate output stage back to the input stage. The active signal indicates to the input stage that its address is being driven on to the chosen slave.

During the data phase of a transfer the decoder routes the appropriate ready signal, transfer response, and read data back to the input port.

The decode stage is intended to be modifiable, so that you can adapt the address map of the BusMatrix to suit your system design. By default, the decoder is supplied with each of the output ports occupying 16MB of address space, that is, **HADDR[26:24]** is used to determine the output port required.

In multi-layer AHB systems that have local slaves on some of the AHB layers, the address decoding is performed in two stages. The first address decode selects between local slaves and the shared slaves available through the BusMatrix. To support this, the decode stage within the BusMatrix includes an **HSEL** input that indicates if the address from an input port is destined for a shared slave.

Output stage

The output stage serves three functions:

- selects the address and control signals from the various input stages
- selects the write data from the appropriate input stage
- determines when to switch between the input stages.

The output stage only selects an input source when that input has a transfer in the holding register.

The output stage generates an active signal for each input port that indicates that the address from that input port is being driving to the slave. This active signal allows the input stage to know when transfers are being routed to the slave and when transfers have to be held because the slave is not available.

When a sequence of transfers to a shared slave has finished and there are no more transfers to the slave required by any of the input ports, the output stage switches the address and control signals to an idle state.

Arbitration

The arbitration within the BusMatrix determines which input port has access to the shared slave and each shared slave has its own arbitration. Different arbitration schemes provide different system characteristics in terms of access latency and overall system performance.

The following arbitration schemes are supported by the slave switch:

- one port always has highest priority
- switch on every transfer if other port waiting.

The following arbitration schemes can also be considered for a particular application. However, these schemes are not supported and might require significant modification to the arbitration section of the switch:

- switch after *n* transfers if other port waiting
- switch if other port waiting for *n* transfers
- switch after *n* cycles if other port waiting
- switch at the end of fixed length bursts.

Locked transfers

Using a multi-layer AHB system requires certain restrictions to be placed on the use of locked transfers to prevent a deadlock situation. It is required that a sequence of locked transfers are all performed to the same slave within the system. A bus master can ensure this restriction is met by ensuring that a locked sequence of transfers remains within a 1KB address region.

To be more precise, if a bus master is to perform two locked transfer sequences to different address regions, it is required that the bus master does not start the second locked transfer sequence until the final data phase of the first locked transfer sequence has completed.

Configuration

All configurations of input and output ports, up to a maximum of eight ports, are supported:

- two to eight input ports are supported
- one to eight output ports are supported
- round-robin or fixed arbitration are supported.

This gives a total of 112 different configurations (7 x 8 x 2).

Any of the configurations can be built from the generic source using a script that is also supplied.

Low-level optimization

No low-level optimization of the BusMatrix is supported. However, the following changes can be made tying off the appropriate unused input signals and relying on the synthesis process to remove the unused logic:

- different address bus widths
- removal of **HPROT** signals
- removal of **HBURST** signals.

Example timing diagrams

This section gives some example transfer sequences for the BusMatrix. For clarity, all transfer sequences shown are read operations, but the same cycle behavior occurs for writes.

Figure 6 shows simultaneous access from both ports with switching occurring every transfer. In the example the shared slave inserts a wait state on the first transfer from each port, but all subsequent transfers are zero wait state.

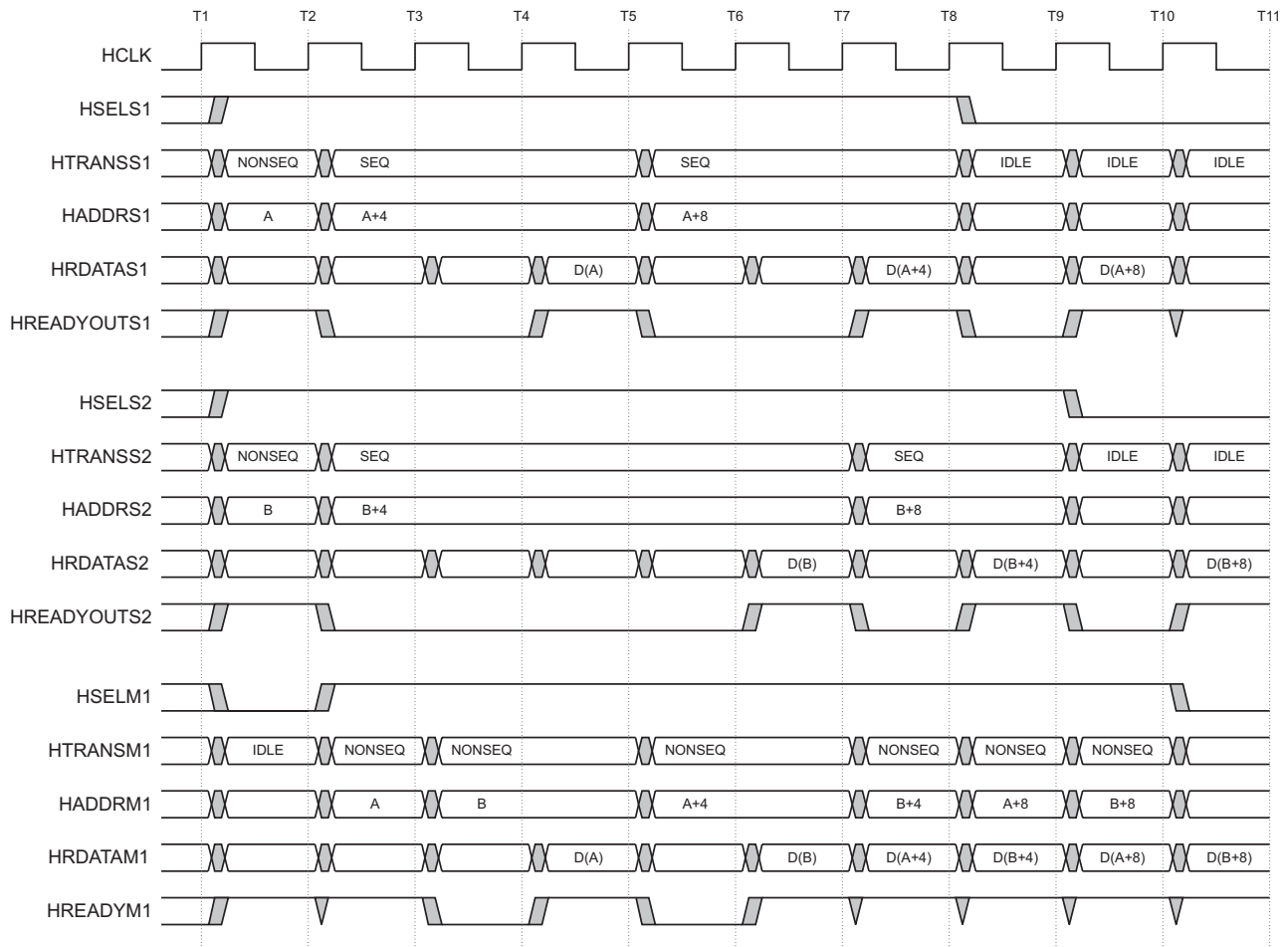


Figure 6 Parallel access timing diagram

Figure 7 shows a port accessing the shared slave when the other input port is idle. At the start of the sequence, a single wait state is inserted by the BusMatrix. This example shows the shared slave inserting one wait state for each of the first two accesses.

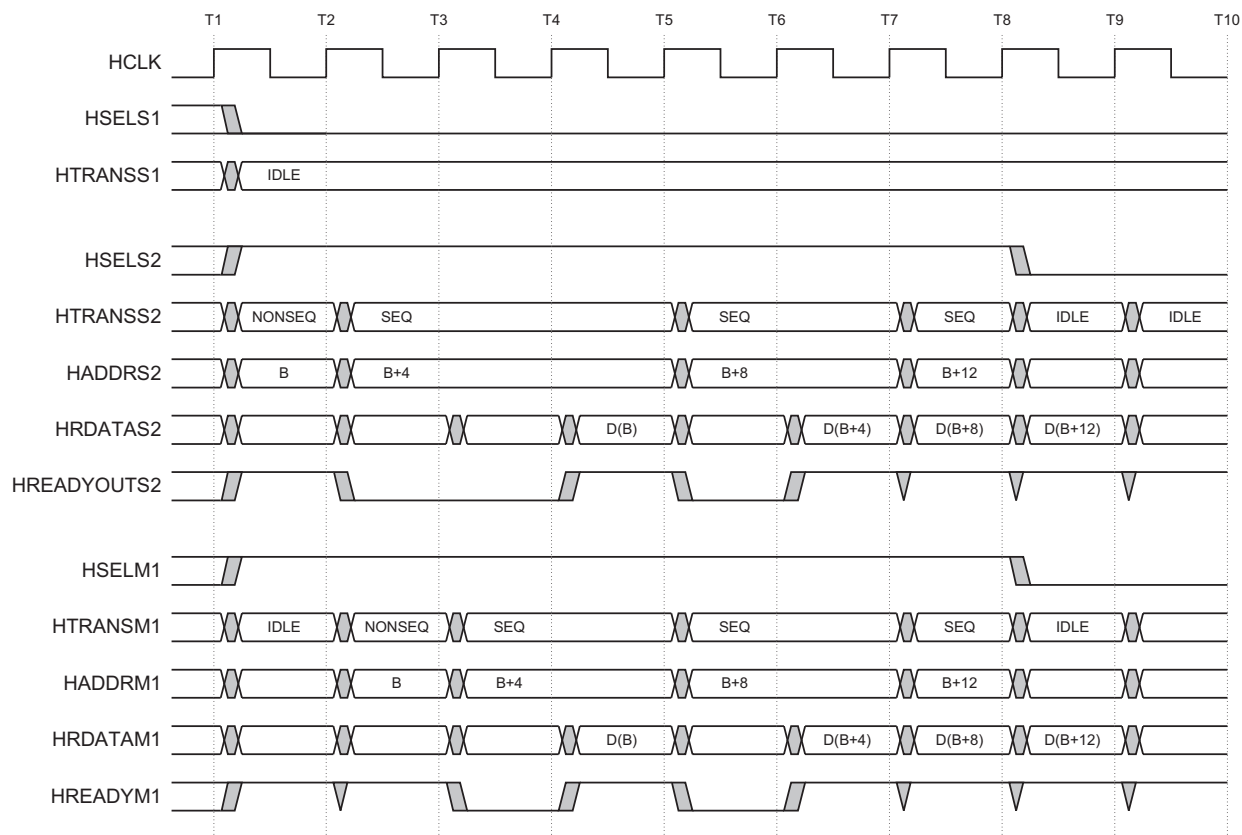


Figure 7 Single port access timing diagram

Interface diagram and signal list

Figure 8 shows the signal interface for the BusMatrix.

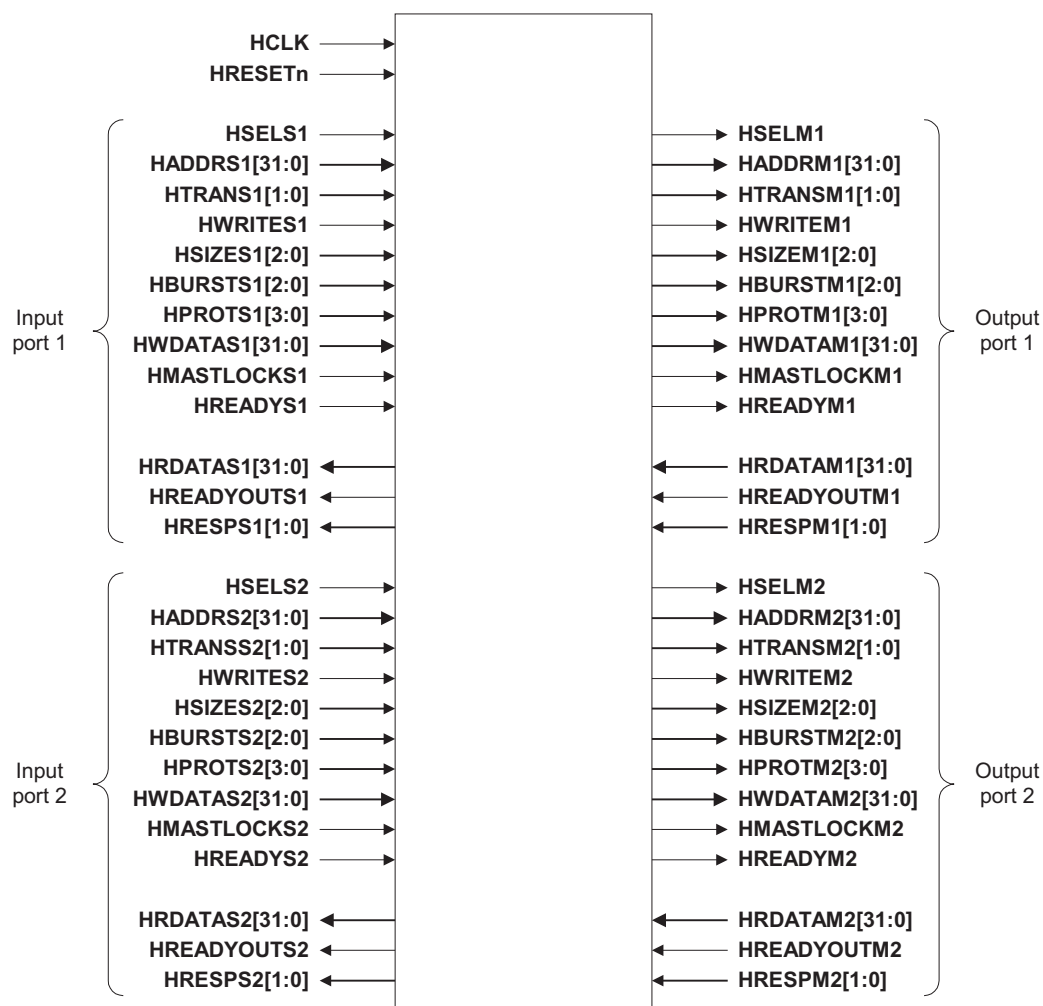


Figure 8 Signal interface diagram

Table 1 shows a list of the interface signals used by the BusMatrix. An identical set of signals can be found on each of the input ports and each of the slave output ports.

Table 1 BusMatrix interface signals

Name	Direction	Description
AMBA signals		
HCLK	Input	Bus clock
HRESETn	Input	Reset
Input port signals		
HSELSx	Input	Input port x select signal
HADDRSx[31:0]	Input	Input port x address
HTRANSx[1:0]	Input	Input port x transfer type
HWRITESx	Input	Input port x transfer direction
HSIZESx[2:0]	Input	Input port x transfer size
HBURSTSx[2:0]	Input	Input port x burst type
HPROTSx[3:0]	Input	Input port x protection control
HWDATASx[31:0]	Input	Input port x write data
HMASTLOCKSx	Input	Input port x locked transfer
HREADYx	Input	Input port x ready signal
HRDATASx[31:0]	Output	Input port x read data
HREADYOUTSx	Output	Input port x ready output
HRESPSx[1:0]	Output	Input port x response
Output port signals		
HSELMx	Output	Output port x select signal
HADDRMx[31:0]	Output	Output port x address
HTRANSMx[1:0]	Output	Output port x transfer type
HWRITEMx	Output	Output port x transfer direction
HSIZEMx[2:0]	Output	Output port x transfer size
HBURSTMx[2:0]	Output	Output port x burst type
HPROTMx[3:0]	Output	Output port x protection control
HWDATAMx[31:0]	Output	Output port x write data
HMASTLOCKMx	Output	Output port x locked transfer
HREADYMx	Output	Output port x ready signal
HRDATAMx[31:0]	Input	Output port x read data
HREADYOUTMx	Input	Output port x ready output
HRESPMx[1:0]	Input	Output port x response

AC characteristics

The BusMatrix conforms to the following AMBA timing parameters:

- combinatorial input to output paths less than 20% of **HCLK** cycle
- all outputs valid 60% of cycle after rising **HCLK**
- registered inputs setup 60% of cycle before rising **HCLK**.

The timing characteristics are confirmed by performing synthesis on the slave switch using the slow/slow process point of the Avanti cb25 cell library at a target speed of 100MHz.

Gate count

The gate count estimate depends on the configuration of the BusMatrix and falls within the following boundaries:

- 2 input ports, 1 output port: 2k gates
- 8 input ports, 8 output port: 25k gates.

Test methodology

The AHB BusMatrix is designed for full scan test insertion. No dedicated test features are included.

